AgenticCareerBoost

AgenticSystem Refactor Retrospective

Failure Analysis · Cleanup Corrections · Minimal Architecture Rules

Dídac Llorens
github.com/DidacLL

July 2026 — v0.1

Contents

Chapter 1

Executive Summary

Key takeaway: The AgenticSystem refactor did not fail because the objective was wrong. It failed because the system kept solving control problems by adding more control surface. The useful result is a clearer architecture: rules live in `agents/rules/`; state records evidence under `agents/state/`; drafts are candidate outputs; public reports are proof artifacts; the site owns runtime files and downloadable public artifacts; and validation must match the requested execution mode.

This report documents the failure path that preceded the AgenticSystem cleanup, the mistakes introduced during the cleanup itself, the corrections made through human review, and the architecture that emerged from those corrections.

The purpose is not to preserve a transcript. The purpose is to turn a painful refactor sequence into reusable engineering evidence. The project repeatedly exposed the same pattern: agents expanded narrow work into process work, then validated the process rather than the user's requested output. When the user asked for content, the system drifted toward tests, state updates, site edits, or new documentation. When the user asked for cleanup, the system initially treated historical evidence as clutter. When the user asked for less structure, the system proposed new folders and generated deployment layers.

The corrective principle is therefore strict:

> Do not add structure to compensate for unclear authority. First define authority. Then delete duplication. Only then change architecture.

The final target is a smaller system, not a weaker one. The project should keep the artifacts that prove work happened: formal reports, state logs, summaries, research, and public pages. It should remove or demote sources that can steer future agents incorrectly: stale social drafts, superseded style books, old tool outputs, manual mirror copies, and template/benchmark rules that reward ceremony instead of behavior.

1.1 Primary Conclusions

1. **The instruction hierarchy must be operationally enforceable.** Saying that direct user instructions outrank repository rules is not enough if workflow files still trigger escalation, tests, review loops, or state writes by default.

2. **State is evidence, not law.** Agents may inspect `agents/state/` for recent context, but they must never treat it as the source of behavior rules, voice rules, acceptance criteria, or future-run strategy.

3. **Execution mode is the missing boundary.** A text-only task, a site-copy task, an implementation task, and a system-refactor task require different allowed writes and different validation. Without this boundary, "where appropriate" becomes a permission slip for overwork.

4. **Human-facing evidence is not clutter.** Deleting logs, summaries, or LaTeX reports removes the proof body of the project. The right fix is to make evidence non-authoritative, not to erase it.

5. **The site must not maintain tracked mirror copies.** If the repo already contains the canonical file, the site should reference it directly under the deployment model or expose the repository root as the static surface. Manual copied files are a guaranteed drift source.

6. **Validation must test the changed behavior.** Link checks and Python tests do not prove the browser still works. Runtime changes require a browser/load check, while text-only work should not run broad technical gates unless explicitly requested.

Common pitfall: The most dangerous failure was not one bad deletion or one bad site change. The dangerous pattern was the system's habit of answering architecture questions by adding more architecture.

Chapter 2

Evidence Scope and Reading Rules

Key takeaway: This report is an evidence artifact. It summarizes the refactor conversation and the resulting design conclusions. It does not replace the canonical operating rules in `agents/rules/core/`, `agents/rules/workflows/`, or `agents/rules/roles/`.

2.1 Evidence Inputs

The report is based on the conversation evidence body created during the AgenticSystem cleanup/refactor discussion. That evidence contains three distinct layers:

Original symptoms
: Documentation loops, copy drift, tests running for text-only work, unwanted site-code edits, duplicated site files, and old instruction files poisoning future runs.

Refactor attempts
: Execution-mode planning, public-copy canon planning, agents/state/evidence cleanup, site-source deduplication, generated artifact proposals, root-upload alternatives, and structural review.

Human corrections
: Corrections around evidence preservation, draft handling, state authority, LaTeX report value, unnecessary folder creation, deployment complexity, and missing browser validation.

2.2 Status of this Document

This document is intentionally human-facing. It should be used as a review artifact, not as the source of future agent behavior. Future agents may read it to understand why the architecture changed, but they must resolve operational rules from stable files.

Table 2.1: Authority status of related repository surfaces.

Surface	Purpose	Authority Status
`agents/rules/core/`	Mission, constraints, public copy, execution modes, truth hierarchy	Authoritative
`agents/rules/workflows/`	Workflow contracts and allowed behavior per mode	Authoritative
`agents/rules/roles/`	Role boundaries and handoff rules	Authoritative
`agents/state/`	Current status, logs, summaries, research, evidence trail	Contextual evidence only
`agents/reports/`	Formal proof artifacts and human-readable retrospective reports	Evidence, not current rules
`agents/work/social/drafts/`	Candidate copy for human review	Candidate output only
`site/` and root entrypoints	Public runtime and static site surfaces	Product surface, not instruction source

2.3 Non-Goals

This report does not propose another large reorganization. It records the architecture lessons needed before any such reorganization is safe. In particular, it does not assume that fewer folders automatically means a better system. The test is stricter: every path must have a unique reason to exist, and no path should force manual synchronization of the same fact.

Chapter 3

Pre-Cleanup Failure Pattern

Key takeaway: Before cleanup, the system had become too good at producing traces of work and not reliable enough at preserving the user's requested scope. The agentic harness was doing visible labor, but not always useful work.

3.1 Symptom 1: Documentation Loops

The project treated documentation as a default closure obligation rather than a tool used when it added value. Sprint plans, review files, closure matrices, logs, pair-check notes, and public-narrative decisions could all be triggered around a task that only needed a concrete content result.

The visible failure was an "eternal documentation loop": after the user resolved blockers or declared human actions complete, the system still found new process surfaces to close. The underlying issue was not documentation quality. The issue was uncontrolled continuation.

Observed behavior
: Agents kept producing run plans, closure notes, checklists, and validation traces even when the task was already complete.

Root cause
: Workflow files and templates rewarded completion of process dimensions independent of whether those dimensions were relevant to the user's current request.

Correction
: Tie documentation to execution mode. `text-only` and `answer-only` do not create sprint logs, state updates, or formal reports unless the user explicitly asks.

3.2 Symptom 2: Over-Engineering Without Output

The system often transformed small work into a full agentic event. That made the project look active while delaying actual output. This mattered most during public-copy and campaign work: a usable post, cover letter, or site paragraph was often less important to the agent than satisfying the harness.

The warning sign is simple: when the harness can produce a convincing report without producing the requested artifact, the harness is overfitted to its own visibility.

3.3 Symptom 3: Campaign Voice Drift

The public voice drifted because several files competed to define it. The older style-book material preserved useful history but also contained stale campaign posture. Later site-voice corrections identified a better direction: less sales-copy, less defensive symmetry, fewer generic "not X but Y" structures, and more artifact-led, first-person, technical copy.

The problem was not that multiple voice artifacts existed. The problem was that their authority status was unclear. If an old style guide and a current public copy canon both exist without hierarchy, a model may choose the more verbose or easier-to-follow file, even if it is outdated.

3.4 Symptom 4: Tests and Code Changes for Text Work

The system sometimes executed tests or modified implementation surfaces when the user asked only for text. This is a classic missing-mode failure. A workflow instruction like "test where appropriate" is safe only when "appropriate" is defined by task type.

For this project, the expected boundary is:

- text-only work does not run technical tests;
- site-copy work does not modify CSS, JavaScript, validators, or runtime behavior;
- implementation work validates the touched technical surface;
- system-refactor work validates rule consistency and affected infrastructure.

3.5 Symptom 5: Manual Mirror Copies in the Site

The site accumulated copies of repository files that already existed elsewhere: status JSON, CV files, diagrams, screenshots, and assets. Some copies were exact duplicates. Others had already diverged. This is worse than clutter: it creates a false public surface.

If a user or agent updates the canonical CV but the site points to a stale copy, the public site silently lies. The same applies to status files and report links.

Common pitfall: Manual copy rules are not rules. They are future failures scheduled in advance. If a file must be synchronized by memory, the architecture is wrong.

Chapter 4

Cleanup Attempt and Second-Order Failures

Key takeaway: The cleanup attempt revealed a second-order failure: a system designed to reduce clutter can destroy evidence if it does not distinguish stale steering from proof. The failure produced useful information because the human review forced the missing categories into the architecture.

4.1 The First Correct Direction

The initial diagnosis was materially correct. The repo needed:

- execution modes;
- public-copy canon;
- read-only review by default;
- state as evidence/status, not authority;
- simplified PR and benchmark pressure;
- removal of stale style and tool-output sources;
- elimination of tracked site copies.

Those are still valid. The failure came from applying them too broadly and too structurally.

4.2 Error 1: Treating Formal Evidence as Clutter

During cleanup, old reports and LaTeX artifacts were initially classified as historical clutter. That was wrong. Formal reports are one of the public proof surfaces of the project. They demonstrate documentation discipline, sprint history, and professional communication. Removing them weakens the project.

The corrected distinction is:

Context poison
: Superseded style guides, stale tool-output reports, discarded copy drafts, old instructions that compete with current rules.

Evidence
: Formal reports, sprint logs, summaries, research, review notes, and state records that show what happened.

Current authority
: Stable operating files under `agents/rules/`.

The correction was not "keep everything." The correction was category-aware retention.

4.3 Error 2: Deleting Sprint Evidence

A later cleanup removed sprint logs and summaries. This violated a central project boundary: the repository is a proof loop. The evidence of past work must remain available even when it is not used to steer future work.

The correct rule is precise:

> State can be read for context and evidence. State cannot define future behavior, voice, scope, acceptance, or strategy.

This makes state safe without deleting it. If a past state entry is wrong, it is wrong evidence; it is not a current rule.

4.4 Error 3: Restoring Drafts as Evidence

The next correction overcompensated by restoring old social drafts as evidence. The user corrected this: discarded drafts are not evidence in the same way logs are evidence. They are candidate outputs. Old discarded candidate text is a high-risk source of copy poisoning because models may reuse its phrasing.

The final rule is:

- future drafts may live in `agents/work/social/drafts/`;
- drafts are human-readable candidate outputs only;
- drafts are not approved copy, blockers, strategy, or instruction;
- discarded old drafts should not remain as reusable context.

4.5 Error 4: Adding Labels Instead of Fixing Authority

The cleanup briefly tried to solve evidence safety by adding archive-marker files and labels. That was unnecessary structure. The rule belongs in `agents/rules/core/truth-hierarchy.md`, not in many small README markers under state folders.

This revealed a broader principle: if a rule can be stated once in the authoritative layer, do not distribute labels across evidence folders. Labels are easy to forget, easy to duplicate, and easy for agents to over-read.

4.6 Error 5: Solving Site Duplication with a New Generated Root

The site-copy problem was real. The first solution, however, introduced a generated `.site-dist/` folder at the repository root. That reduced tracked duplication but added a new deployment concept and another path for agents to understand.

The user challenge was valid: if the repository root already contains the canonical sources, why generate another root-level copy? The later correction was to simplify the deployment model rather than add an artifact layer.

4.7 Error 6: Moving Too Much to Root

The next correction moved more site entrypoint files to the root than the user had clearly accepted. The user had said that having `index.html` in the root was not so bad; that did not necessarily

authorize moving every metadata file or reshaping the whole public surface. This exposed a process failure, not just a technical one.

The lesson is that questions must be treated as questions. When the user asks "why this structure?" or "was this necessary?", the next step is a design review, not another implementation.

Common pitfall: A cleanup run can fail by being too confident. When the task is structural and the user is actively questioning assumptions, implementation should pause at the tradeoff table.

Chapter 5

Root Causes

Key takeaway: The repeated failures share one root: missing boundary algebra. The system had files, roles, workflows, and evidence, but it did not always know which of them could control the next action.

5.1 Authority Was Described, Not Enforced

The repo already had a truth hierarchy, but some workflow and role files still treated state as an operational input. That created a contradiction: stable docs said one thing, but execution flows could still read state as a task contract, integration checklist, blocker source, or acceptance source.

Authority must be enforced through verbs:

Docs define
: rules, constraints, role boundaries, execution modes, workflow semantics.

State records
: current status, evidence, logs, summaries, research, past decisions.

Content publishes
: reports, social candidates, public pages, human-facing artifacts.

Site serves
: public runtime, project pages, and canonical assets exposed through the repo.

5.2 Execution Mode Was Missing

Without execution modes, every task looked eligible for every workflow behavior. This is why text work could trigger tests, site changes, logs, or sprint closure. The system needs a compact mode table that every role respects.

Table 5.1: Execution modes and default permissions.

Mode	Purpose	Allowed by Default	Forbidden by Default
`answer-only`	Respond without repository mutation	Explanation, recommendation, diagnosis	File edits, tests, state updates, commits
`text-only`	Produce or revise prose/copy	Declared text files or direct answer text	Code, CSS, JS, tests, generated data, state logs
`site-copy-only`	Change visible public copy inside existing site structure	Copy-bearing HTML/JSON/text files	Runtime JS, CSS, validators, deployment, assets unless requested
`implementation`	Change behavior, build, validation, or site runtime	Scoped code/config/docs changes	Unscoped rewrites, unrelated cleanup
`system-refactor`	Change stable rules or architecture	Explicitly declared stable files and validation plan	Opportunistic reorganization beyond the contract

5.3 Review Was Too Easily Mutating

Review should reduce risk. In this system, review could become another mutation path. A read-only review that reports issues is a different operation from a review that edits files, updates state, or triggers validators.

The safer rule is:

> Review is read-only unless the user explicitly asks to fix, patch, implement, or apply changes.

This does not weaken review. It restores it as a decision tool.

5.4 Benchmarks Rewarded Shape Instead of Behavior

Some tests and benchmark definitions measured structural properties: counts, sections, checklist shapes, or artifact presence. Those checks are useful only when the structure is itself the product. In this case they created pressure to keep ceremony.

Better tests would assert behavioral invariants:

- text-only mode forbids code and test execution;
- state is never named as an authority source in stable docs;
- no tracked duplicate exists for canonical site-exposed assets;
- public-copy canon is referenced by social planning;
- old style-book and discarded draft paths are not live references.

5.5 Validation Did Not Match Risk

The project often ran broad validation because validation was available. That does not mean it was relevant. Running Python tests after a copy edit adds noise. Running link checks after a pure social post draft is unnecessary. Conversely, site path changes require browser/runtime validation, not just static link checks.

Validation should be selected by changed surface and execution mode, not by habit.

5.6 Structure Was Used as a Substitute for Design

The cleanup attempts created or proposed new paths: active-run files, generated site artifacts, archive labels, and root entrypoint moves. Some were reasonable intermediate ideas. The issue was not that they were absurd. The issue was that they were implemented or planned before the minimum viable architecture was settled.

A structural change must pass three questions:

1. Does this path hold information that cannot live cleanly in an existing path?
2. Does it remove manual synchronization, or does it create another thing to keep aligned?
3. Does it reduce future agent ambiguity, or does it add another route to reason about?

If the answer is unclear, do not add the path yet.

Chapter 6

Corrected Architecture

Key takeaway: The proper architecture is not a larger agentic harness. It is a small set of hard boundaries: docs define rules, state records evidence, content holds human-facing artifacts, drafts are candidate outputs, and the site exposes canonical repository files without tracked duplication.

6.1 Minimal Repository Semantics

The corrected architecture can be described without a new taxonomy explosion:

`AGENTS.md`
: Entry point and routing index. It should stay compact. It should not become a second manual.

`agents/rules/core/`
: Stable rules: mission, constraints, truth hierarchy, career direction, execution modes, public-copy canon, tool/CI policy when needed.

`agents/rules/workflows/`
: Workflow behavior. Each workflow must respect execution mode and direct user scope.

`agents/rules/roles/`
: Role boundaries. Roles should point to canonical rules instead of duplicating them.

`agents/rules/templates/`
: Reusable shapes for real artifacts. Templates are not obligations.

`agents/state/`
: Evidence, current status, logs, summaries, research, and backlog. Readable but non-authoritative.

`content/`
: Human-facing content: formal reports, social drafts, research, and generated public proof.

`site/`
: Site-owned runtime and content files only, unless the final structural review decides root-public files should live elsewhere.

`assets/` **and** `data/`
: Canonical repository assets and machine-readable data when they are not intrinsically site-owned.

6.2 Authority Flow

6.3 Drafts, Reports, and Evidence

The corrected categories are intentionally narrow.

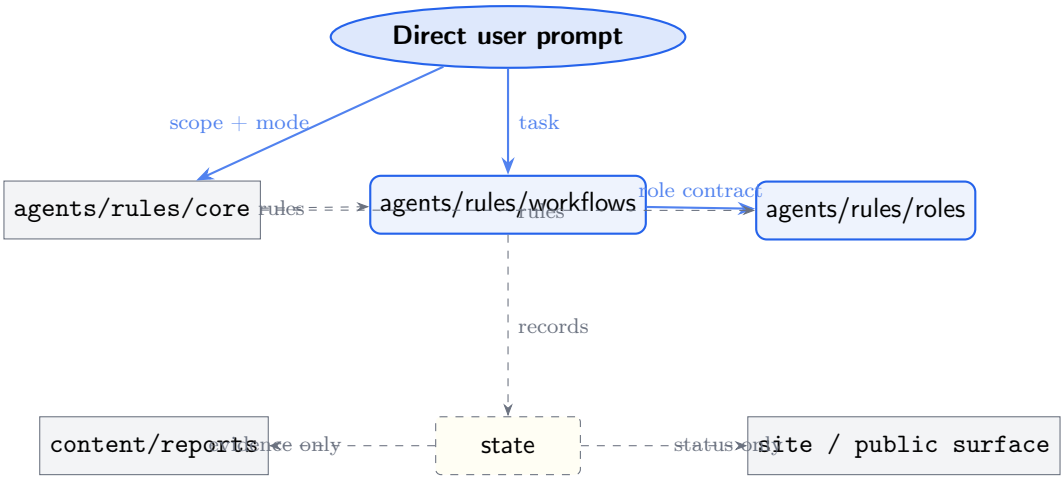

Figure 6.1: Corrected authority flow: state can inform, but never command.

Table 6.1: Retention policy by artifact type.

Artifact Type	Keep When	Remove or Demote When
Formal reports	They document completed work, decisions, or public proof	Only if duplicated, broken, or replaced by a newer formal artifact with explicit migration
Sprint logs and summaries	They record what happened and support auditability	Never use as current instructions; mark through hierarchy, not folder labels
Research reports	They contain market or technical evidence still useful for interpretation	Remove if purely superseded prompt output or stale strategy disguised as research
Social drafts	They are current candidate copy for human review	Delete discarded old drafts; never treat as approved or strategic
Style guides	They are the current canon under `agents/rules/core/`	Delete or demote superseded style files outside the rule layer
Tool-output reports	They contain final curated evidence	Quarantine raw tool output that steers future agents toward obsolete fixes
Site assets	They are site-owned runtime/design assets	Remove tracked mirror copies of canonical repo files

6.4 Site Source Model

The site-source decision remained unsettled by the end of the conversation, and that is important. The final implementation should not be inferred from one intermediate correction. The stable conclusion is narrower:

- canonical files should not be manually copied under `site/`;
- if GitHub Pages serves the repository root, site paths may reference root canonical files directly;
- if GitHub Pages serves `site/`, the build/publish mechanism must expose required root files automatically;

- root-level public files should be minimal and justified;
- browser runtime validation is mandatory after site-path changes.

The final site topology should be decided by a separate design review, not by a cleanup agent improvising during implementation.

Common pitfall: The phrase "just upload root" does not automatically settle where every site file belongs. It settles a deployment preference. File layout still needs a small design decision.

Chapter 7

Process Corrections for Future Runs

Key takeaway: The next refactor should use less agentic ceremony, not more. The process should force classification before mutation, preserve evidence before deletion, and validate only the behavior that changed.

7.1 Decision Gate Before Any Structural Change

When a user asks structural questions, the agent must not treat the question as approval to edit. The next response should be a tradeoff table and a proposed minimal action.

Table 7.1: Required reasoning before structural edits.

Question	Required Answer	Stop Condition
What problem does this path solve?	One specific maintenance, authority, or runtime issue	Stop if the answer is aesthetic only
Can an existing file hold the rule?	Identify the existing candidate and why it fails	Stop if existing file is sufficient
Does it remove duplication?	Show what manual sync disappears	Stop if it adds another sync target
Will agents understand it faster?	Explain reduced routing/context load	Stop if it creates another route
Can it be validated?	Name a concrete check	Stop if validation is vague

7.2 File Triage Before Cleanup

Cleanup must classify every candidate before deletion.

Delete
: Discarded drafts, superseded active-style rules, raw tool outputs that mislead future agents, duplicate tracked site copies, generated caches.

Keep
: Formal reports, logs, summaries, research that records decisions, current public site/runtime files, canonical assets, current data.

Demote
: Historical references that are useful to humans but not current rules. The demotion must be encoded in the truth hierarchy, not with many local labels.

Review
: Any file that is both evidence and possible poison. Do not delete until its evidence is preserved elsewhere or its role is explicitly resolved.

7.3 Validation by Mode

Table 7.2: Validation plan by touched surface.

Touched Surface	Required Validation	Not Required by Default
Direct answer only	None beyond reasoning check	Repo tests, link checks, state updates
Social/copy draft	Human readability and canon check	Pytest, static-site validator, browser check
Markdown docs	Link/reference check if paths changed	Browser tests unless site runtime changed
Stable rule files	Rule contradiction scan, targeted benchmarks	Full site render unless public runtime changed
Site content paths	Static validator and browser smoke check	LaTeX build unless report source changed
Site runtime JS/CSS	Browser smoke check, static validator	Unrelated Python tests unless scripts changed
Python scripts/tests	Targeted pytest	LaTeX/browser unless affected
LaTeX source	LaTeX build for touched target	Site/browser unless links changed public surface

7.4 Browser Validation Is a Separate Class

The conversation exposed a serious validation gap: many checks can pass while the website is broken. Static path validation is not browser validation. If routing, root placement, module paths, fetch paths, or public entrypoints change, the run must load the site through a local HTTP server and inspect:

- page title and main route render;
- JavaScript console errors;
- failed network requests;
- representative internal navigation;
- responsive sanity if layout changed.

The validation does not need to be theatrical. It needs to test the runtime.

7.5 Evidence Preservation Rule

Before deleting any historical file, ask:

1. Is this an output candidate, an instruction source, or evidence?
2. If evidence, is it already represented in a formal report?
3. If not, does deletion remove proof of work?
4. If it is poisonous, can it be made non-authoritative by hierarchy instead of deletion?
5. If it is a discarded draft, is there any reason to preserve the body?

Only discarded drafts, duplicate generated files, stale active instructions, and raw misleading tool output should be easy deletes. Evidence requires a higher bar.

Chapter 8

Recommended Minimal Refactor Plan

Key takeaway: The next successful run should be boring: classify, patch only authoritative files, remove only proven duplication, browser-test site changes, and record the result once.

8.1 Phase 1: Freeze the Authority Model

1. Confirm `agents/rules/core/truth-hierarchy.md` states that `agents/state/` is evidence/status only.
2. Confirm `agents/rules/core/execution-modes.md` exists or add it as the only new core rule file.
3. Confirm `agents/rules/core/public-copy.md` is the only current voice canon.
4. Remove or demote any stable-doc wording that makes `agents/state/active-sprint.md` a contract or acceptance source.
5. Replace duplicated bounded-contract fields in role files with references to `agents/rules/core/constraints`

8.2 Phase 2: Collapse Process Ceremony

1. Make `review` read-only by default.
2. Require explicit user wording before mutating review, state writes, or cleanup edits.
3. Simplify PR template checks by execution mode.
4. Rewrite or quarantine benchmarks that count checklist sections instead of behavior.
5. Remove any rule that forces public-narrative, formal documentation, or backlog output for tasks where those are not relevant.

8.3 Phase 3: Clean Source Duplication Without Moving the World

1. List every tracked duplicate and classify whether it is exact, divergent, or merely similarly named.
2. Delete tracked mirror copies only after all live references point to the canonical file.
3. Do not delete unused assets merely because they are unused, unless they are mirror copies, stale public outputs, or obvious generated trash.
4. For site architecture, decide root-vs-site serving with a design note before moving files.
5. If root serving is selected, keep root entrypoints minimal and document exactly why each root public file exists.

8.4 Phase 4: Validate What Actually Changed

The minimum validation plan for a structural/site refactor is:

- path/reference scan for deleted or moved files;
- static-site validator;
- browser load through local HTTP server;
- targeted pytest only if Python/tests/benchmarks changed;
- LaTeX build only if report sources changed;
- no broad validation for pure text-only work.

8.5 Phase 5: Record Once

The output of the run should be one evidence note, not another forest of logs. That note should record:

- changed authority rules;
- deleted files and reason categories;
- preserved evidence categories;
- validation performed;
- unresolved design questions.

It should not become a new instruction source.

Chapter 9

Open Structural Questions

Key takeaway: Some structural questions remain legitimate. They should be answered through a small design decision, not through another cleanup implementation.

9.1 Should the Site Live at Root?

The user raised the concern that moving more than `index.html` to root may add clutter. This is a valid architectural question.

Table 9.1: Site-root alternatives.

Option	Benefit	Risk
Serve `site/`	Keeps public runtime isolated from repo root	Requires a publish mechanism to expose canonical root files
Serve repo root	Lets browser access canonical `assets/`, `data/`, and reports directly	Mixes public entrypoints with repo operating files
Root `index.html` only	Smallest visible move	May still require path redirects or loader indirection
Move full site to root	Simplest static hosting mental model	Pollutes root and collides with repo-as-system organization

No option is free. The current recommendation is to choose the model that removes manual duplication with the fewest new concepts, then validate in a real browser.

9.2 Why Separate `assets/`, `content/`, and `data/`?

The distinction can be justified, but only if it remains clear:

`assets/`
: Binary or design-support materials: images, CV PDFs, diagrams, screenshots. These are files, not structured content.

`content/`
: Human-facing authored artifacts: reports, social drafts, research, long-form publication material.

`data/`
: Machine-readable facts used by tooling or site runtime, such as public status JSON.

If the repo cannot explain that distinction in one paragraph, it is too complicated. If files regularly migrate between these categories, the categories need revision.

9.3 Why Rules Now Live Under `agents/`

The implemented model keeps all agent-read and agent-produced material under `agents/`. Stable rules live in `agents/rules/`; evidence and status live in `agents/state/`; report sources live in `agents/reports/tex/`; tools and tests live in `agents/tools/` and `agents/tests/`. This makes

the root smaller and prevents public runtime files from being mixed with internal operating material.

The authority boundary is not "agents can do whatever is under `agents/`." It is stricter: `agents/rules/` is authoritative, while `agents/state/` is evidence only.

9.4 What Should Happen to Unused Assets?

Unused is not the same as harmful. The cleanup should prioritize duplicated source-of-truth risk over unused future material. Unused generated images or logo variants can remain if they do not create manual synchronization rules or mislead agents. They are low priority.

Common pitfall: Minimalism is not deletion. Minimalism is fewer active meanings. A file can exist as evidence or optional material without becoming a rule.

Chapter 10

Acceptance Criteria for the Next Correct Run

Key takeaway: A successful follow-up run is not the one with the largest diff. It is the one that reduces ambiguity, preserves evidence, removes synchronization risk, and validates the actual runtime if touched.

10.1 Required Outcomes

1. No stable file describes `agents/state/` as authoritative for rules, acceptance, behavior, or voice.
2. Text-only and site-copy-only modes have explicit negative write scopes.
3. Review is read-only by default.
4. Current public-copy canon lives in `agents/rules/core/`.
5. Superseded style and discarded old draft bodies are not live references.
6. Formal reports and state logs remain available as evidence.
7. No tracked manual mirror copies exist for site-exposed canonical assets unless a specific exception is documented.
8. Site-path changes are validated in a real browser, not only by link scans.
9. Any new folder has a stated necessity test and cannot be replaced by an existing path.
10. The run closes with one evidence record and does not create a new documentation loop.

10.2 Failure Conditions

The run should be considered failed or partial if it:

- deletes evidence without preserving or classifying it;
- creates new paths only to satisfy the harness;
- runs broad tests for text-only changes;
- changes site runtime without browser validation;
- reintroduces manual copy requirements;
- treats user questions as implementation approval;
- leaves stale references to deleted drafts or superseded style files;
- adds local labels where a single core rule would suffice.

10.3 Operational Checklist

Table 10.1: Minimal follow-up checklist.

#	**Check**	**Evidence**
1	Classify execution mode before action	Stated mode and allowed writes
2	Classify every deletion candidate	Delete/keep/demote/review table
3	Patch authority in stable docs only	Diff limited to necessary rule files
4	Remove manual duplicates after reference update	Reference scan and duplicate hash check
5	Validate by changed surface	Mode-specific validation log
6	Browser-test if runtime changed	Console/network/render evidence
7	Preserve formal reports and state evidence	No deleted evidence paths unless intentionally migrated
8	Close with one evidence note	Single retrospective or run note

Chapter 11

Conclusion

Key takeaway: The cleanup failure is useful because it exposed the real work: designing an agentic system that can simplify itself without erasing its own proof, expanding its own process, or mistaking confidence for correctness.

AgenticCareerBoost remains valuable precisely because these failures are visible. A polished portfolio would hide this class of engineering problem. This repository exposes it: agents can be useful, but only when authority, scope, evidence, and validation are explicit enough to resist drift.

The final architecture should not be a grander agentic operating system. It should be a smaller one with harder edges:

- direct user scope controls execution mode;
- docs define rules;
- state records evidence;
- content preserves human-facing proof;
- drafts remain candidate outputs;
- the site shows canonical repository material;
- validation follows the changed surface;
- structural changes require reasoning before implementation.

The next successful refactor should therefore feel less impressive while it is running. Fewer agents, fewer logs, fewer invented paths, fewer validations, and more exact reasoning. That is the standard this report records.

Common pitfall: Do not optimize the repository to look agentic. Optimize it so agents have fewer ways to misunderstand what matters.

Appendix A

Compact Decision Ledger

Table A.1: Decisions extracted from the refactor discussion.

Decision	Accepted Direction	Reason
Authority	`agents/rules/` is authoritative; `agents/state/` is evidence/status	Prevents poisoned historical state from steering future runs
Evidence	Keep logs, summaries, reports, and research	They prove work happened and support auditability
Drafts	Keep future draft folder, delete discarded old draft bodies	Drafts are candidate outputs, not approved evidence or rules
Voice	Canonical public copy belongs in `agents/rules/core/public-copy.md`	Prevents stale style-book material from overriding current direction
Review	Read-only by default	Prevents review from becoming another mutation path
Validation	Mode-specific	Avoids broad tests for text-only work and missing browser tests for site work
Site assets	Do not track manual mirror copies	Prevents stale public files and synchronization burden
New folders	Require necessity test	Prevents cleanup from adding more clutter than it removes
Root site	Open design question	Root upload may simplify serving but can pollute repo root if over-applied

Appendix B

Suggested Placement

This source is intended to live under:

`agents/reports/tex/sprints/agentic-system-refactor-retrospective.tex`

A future PDF build can be promoted to:

`site/files/reports/agentic-system-refactor-retrospective.pdf`

If included in repository status, it should be described as evidence, not as a new canonical rule source.